Simulador de un Sistema Bancario Concurrente Avanzado

SecureBank — Memoria Técnica

Asignatura: Sistemas Operativos
Universidad Francisco de Vitoria — UFV Madrid

Autores:

- Mario Gayarre Tolivar
- Rodrigo Torre Santamaría
- Marco Aurelio Cabral de Pablo

Fecha: Abril 2026

Índice

1. Introducción
2. Arquitectura general del sistema
3. Descripción de los componentes
   - 3.1 banco_comun.h — Cabecera compartida
   - 3.2 banco.c — Proceso principal
   - 3.3 usuario.c — Proceso hijo (usuario)
   - 3.4 monitor.c — Proceso de vigilancia
4. Mecanismos de IPC y sincronización
   - 4.1 Semáforos con nombre POSIX
   - 4.2 Colas de mensajes POSIX
   - 4.3 Pipes anónimos

- 4.4 Señales
- 4.5 Threads POSIX (pthreads)

5. Flujo de ejecución completo
6. Detección de anomalías
7. Fichero de configuración
8. Compilación y arranque
9. Capturas de funcionamiento
10. Cuaderno de bitácora

---

1. Introducción

SecureBank es un simulador de un sistema bancario concurrente implementado en C sobre Linux. El objetivo del proyecto es demostrar el dominio de las características avanzadas del sistema operativo: gestión de procesos, comunicación interproceso (IPC), sincronización con semáforos, colas de mensajes POSIX y manejo de señales, todo ello aplicado a un caso de uso real.

El sistema gestiona múltiples usuarios simultáneos, cada uno representado por un proceso hijo independiente. Dentro de cada proceso hijo, las operaciones bancarias se delegan a *threads*, garantizando la exclusión mutua al acceder a los ficheros compartidos mediante semáforos. Un proceso monitor independiente analiza el tráfico de operaciones en tiempo real y emite alertas cuando detecta patrones sospechosos.

---

2. Arquitectura general del sistema

```
┌──────────────────────────────────────────────────────────────┐
│                          banco.c                             │
│                (Proceso Principal / Padre)                   │
│                                                              │
│  • Menú de login (nº de cuenta / crear cuenta)              │
│  • Crea y destruye semáforos                                 │
│  • Crea las 3 colas POSIX (MQ_LOG, MQ_ALERTA,              │
│    MQ_MONITOR)                                               │
│  • Lanza monitor (fork+execv)                                │
│  • Por cada login: fork+execv → usuario                      │
│  • bucle poll(): atiende MQ_LOG, MQ_ALERTA y stdin           │
└──────────────────────────────────────────────────────────────┘
         │              │                  │
         │ fork+execv   │ fork+execv       │ fork+execv
         │              │                  │
         ▼              ▼                  ▼

  ┌────────────┐  ┌────────────┐   ┌────────────┐
  │ usuario.c  │  │ usuario.c  │   │ monitor.c  │
  │ (hijo 1)   │  │ (hijo N)   │   │            │
  │            │  │            │   │ Lee        │
  │ Menú       │  │ Menú       │   │ MQ_MONITOR │
  │ interactivo│  │ interactivo│   │ Detecta    │
  │            │  │            │   │ patrones   │
  │ threads:   │  │ threads:   │   │ Envía a    │
  │ depósito   │  │ depósito   │   │ MQ_ALERTA  │
  │ retiro     │  │ retiro     │   └────────────┘
  │ trf.       │  │ trf.       │
  │ divisas    │  │ divisas    │
  └──────┬─────┘  └──────┬─────┘
         │               │
         │ MQ_LOG        │ MQ_LOG        ← mensajes de log al padre
         │ MQ_MONITOR    │ MQ_MONITOR    ← datos para el monitor
         │               │
         │ pipe rd ◄─────┼── banco.c escribe "BLOQUEO" si procede
         │               │
         ▼               ▼
   cuentas.dat (fichero binario compartido, protegido por /sem_cuentas)
   config.txt  (PROXIMO_ID, protegido por /sem_config)
   transacciones.log (escrito solo por banco.c)
```

Resumen de procesos y roles

| Proceso | Creado por | Rol | |---------|----------|-----| | `banco` | el shell | Coordinador. Login, gestión de cuentas, escritura del log | | `usuario` (N instancias) | `banco` vía fork+execv | Sesión interactiva de un usuario. Lanza threads de operación | |

`monitor` (1 instancia) | `banco` vía fork+execv | Vigilancia de anomalías en tiempo real |

---

3. Descripción de los componentes

3.1 `banco_comun.h` — Cabecera compartida

Centraliza en un único fichero de cabecera todos los elementos que comparten los tres módulos, evitando duplicación de código:

Constantes del sistema:

- `ID_INICIAL 1001` — número de cuenta inicial, usado como base de offset en `fseek()`.
- `MAX_TITULARES`, `MAX_PATH`, `MAX_TS`, `MAX_ALERT` — límites de tamaño de cadenas.
- Tipos de operación: `OP_DEPOSITO`, `OP_RETIRO`, `OP_TRANSFERENCIA`, `OP_MOVER_DIVISA`.
- Divisas: `DIV_EUR (0)`, `DIV_USD (1)`, `DIV_GBP (2)`.
- Nombres de colas POSIX: `MQ_MONITOR`, `MQ_LOG`, `MQ_ALERTA`.
- Nombres de semáforos: `SEM_CUENTAS ("/sem_cuentas")`, `SEM_CONFIG ("/sem_config")`.

Estructuras de datos:

```
typedef struct {
    int   numero_cuenta;
    char  titular[MAX_TITULARES];
    float saldo_eur;
    float saldo_usd;
    float saldo_gbp;
} Cuenta;
```

Unidad de almacenamiento en el fichero binario `cuentas.dat`. El tamaño fijo de la estructura permite acceso directo mediante `fseek()`.

```
typedef struct {
    int   cuenta_origen;
    int   cuenta_destino;   /* 0 si no es transferencia */
    int   tipo_op;
    float cantidad;
    int   divisa;
    char  timestamp[MAX_TS];
} DatosMonitor;
```

Mensaje enviado por los hijos al monitor a través de `MQ_MONITOR`.

```
typedef struct {
    int    cuenta_id;
    pid_t  pid_hijo;
    int    tipo_op;
    float  cantidad;
    int    divisa;
    int    estado;          /* 0 = OK, 1 = fallo */
    char   timestamp[MAX_TS];
} DatosLog;
```

Mensaje enviado por los hijos al padre a través de `MQ_LOG` para generar las entradas del log.

```
typedef struct {
    int  cuenta_id;
    char mensaje[MAX_ALERT];
} DatosAlerta;
```

Mensaje enviado por el monitor al padre a través de `MQ_ALERTA`.

```
typedef struct {
    int   proximo_id;
    float lim_ret_eur, lim_ret_usd, lim_ret_gbp;
    float lim_trf_eur, lim_trf_usd, lim_trf_gbp;
    int   umbral_retiros;
    int   umbral_transferencias;
    int   num_hilos;
    char  archivo_cuentas[MAX_PATH];
    char  archivo_log[MAX_PATH];
    float cambio_usd;
    float cambio_gbp;
} Config;
```

Configuración completa leída de `config.txt`.

Funciones utilitarias inline:

- `timestamp_ahora(buf, n)` — rellena `buf` con la fecha y hora actual en formato `YYYY-MM-DD HH:MM:SS` usando `strftime`.
- `nombre_divisa(d)` — devuelve la cadena `"EUR"`, `"USD"` o `"GBP"` a partir del índice de divisa.

3.2 `banco.c` — Proceso principal

Es el programa de entrada del sistema. Su función `main()` ejecuta las siguientes fases en orden:

Inicialización

1. Registra manejadores de señal: `SIGTERM` y `SIGINT` activan la variable `g_salir`; `SIGCHLD` se deja en `SIG_DFL` para poder usar `waitpid`.
2. Lee `config.txt` con `leer_config()`.
3. Destruye y vuelve a crear los semáforos con nombre `/sem_cuentas` y `/sem_config` (valor inicial = 1, modo mutex).
4. Crea el fichero `cuentas.dat` si no existe.
5. Destruye y vuelve a crear las tres colas POSIX en modo no bloqueante (`O_NONBLOCK`), con el tamaño de mensaje ajustado a cada estructura:
   - `MQ_MONITOR` → `sizeof(DatosMonitor)`
   - `MQ_LOG` → `sizeof(DatosLog)`

- `MQ_ALERTA` → `sizeof(DatosAlerta)`

6. Lanza el proceso monitor mediante `lanzar_monitor()`.

Funciones internas de `banco.c`

`leer_config(ruta, cfg)`
Abre `config.txt` en modo lectura y parsea línea a línea con `sscanf("%[^=]=%s")`, ignorando comentarios (`#`) y líneas vacías. Mapea cada clave al campo correspondiente de la estructura `Config`.

`guardar_proximo_id(nuevo_id)`
Actualiza la línea `PROXIMO_ID=` en `config.txt` sin reescribir el fichero completo desde cero: lee todo el contenido en memoria, localiza la cadena `"PROXIMO_ID="` con `strstr`, construye el fichero nuevo con `snprintf` y lo sobreescribe. Se llama siempre dentro de la sección crítica de `SEM_CONFIG`.

`buscar_cuenta(numero, c)`
Búsqueda lineal en `cuentas.dat`: abre el fichero en modo `"rb"` y recorre las estructuras `Cuenta` con `fread` hasta encontrar `numero_cuenta == numero`.

`crear_cuenta(nueva)`
Opera en dos secciones críticas anidadas pero separadas:

1. Bajo `/sem_config`: lee `PROXIMO_ID`, lo asigna a `nueva->numero_cuenta`, lo incrementa y persiste en disco.
2. Bajo `/sem_cuentas`: abre `cuentas.dat` en modo `"ab"` y añade la estructura al final con `fwrite`.

`escribir_log(linea)`
Abre `transacciones.log` en modo `"a"` (append) y escribe la cadena recibida seguida de un salto de línea.

`buscar_hijo(cuenta_id)`
Recorre el array estático `g_hijos[]` y devuelve un puntero a la entrada cuyo `cuenta_id` coincida.

`procesar_log()`
Drena la cola `MQ_LOG` en un bucle `while(mq_receive(...) > 0)`. Por cada mensaje construye una línea de log con el formato `[timestamp] Operacion en cuenta X: Y.YY DIV - OK/FALLIDO` y la pasa a `escribir_log()`.

`procesar_alertas()`

Drena `MQ_ALERTA`. Por cada alerta:

1. Determina si procede bloqueo buscando las cadenas `ALERTA_RETIROS` o `ALERTA_TRANSFERENCIAS` en el mensaje con `strstr`.
2. Escribe la entrada en el log.
3. Si procede bloqueo, localiza el hijo afectado con `buscar_hijo()` y escribe `"BLOQUEO: ..."` en el extremo de escritura del pipe correspondiente, cerrándolo a continuación.

`lanzar_usuario(cuenta_id, pipe_wr_out)`

1. Crea un pipe anónimo con `pipe(pfd)`.
2. Hace `fork()`.
   - **Hijo**: cierra el extremo de escritura (`pfd[1]`), cierra los descriptores de las colas del padre y llama a `execv("./usuario", args)` pasando `cuenta_id` y `pfd[0]` como argumentos de cadena.
   - **Padre**: cierra el extremo de lectura (`pfd[0]`), devuelve el extremo de escritura (`pfd[1]`) a través de `pipe_wr_out`.

`lanzar_monitor()`

Análoga a `lanzar_usuario()` pero sin pipe: hace `fork()` y el hijo llama a `execv("./monitor", args)`. El padre guarda el PID del monitor en `g_monitor_pid` para poder enviarle `SIGTERM` al cierre.

Bucle principal (login y sesión)

El proceso padre usa `poll()` sobre tres descriptores simultáneamente:

- `STDIN_FILENO` — para leer el número de cuenta introducido por el usuario.
- `g_mq_log` — para procesar entradas de log pendientes.
- `g_mq_alerta` — para procesar alertas del monitor.

Cuando llega entrada por `stdin`:

- Si `numero == 0`: crea una cuenta nueva con `crear_cuenta()`.
- Si `numero > 0`: busca la cuenta con `buscar_cuenta()`.
- En ambos casos lanza el proceso hijo con `lanzar_usuario()` y entra en un sub-bucle de sesión con `poll()` sobre `MQ_LOG` y `MQ_ALERTA`, con `waitpid(pid, WNOHANG)` para detectar la terminación del hijo.

Cierre del sistema

Cuando `g_salir == 1`:

1. Cierra el extremo de escritura de todos los pipes de hijos activos.
2. Espera a todos los hijos con `waitpid`.
3. Envía `SIGTERM` al monitor y espera su terminación.
4. Cierra y destruye las tres colas POSIX.
5. Destruye los dos semáforos con nombre.

3.3 `usuario.c` — Proceso hijo (usuario)

Recibe dos argumentos al ser lanzado por `execv`: `cuenta_id` (número de cuenta) y `pipe_rd` (descriptor de lectura del pipe padre→hijo).

Inicialización del proceso hijo

1. Lee `config.txt` con `leer_config_usuario()`, extrayendo únicamente los límites de operación, tipos de cambio y ruta del fichero de cuentas.
2. Abre las colas `MQ_LOG` (modo `O_WRONLY`) y `MQ_MONITOR` (modo `O_WRONLY`), que ya existen porque las creó `banco.c`.
3. Verifica que el semáforo `/sem_cuentas` está disponible con `sem_open(..., 0)`.

Bucle de eventos con `poll()`

El proceso hijo vigila dos descriptores simultáneamente:

- `STDIN_FILENO` — entrada del usuario por teclado.
- `g_pipe_rd` — pipe del padre, desde el que pueden llegar mensajes de bloqueo.

Cuando llega un evento en el pipe (`POLLIN` en `fds[1]`):

- Lee la cadena con `read()`.
- La muestra por pantalla como `[ALERTA DEL BANCO]: ...`.
- Si contiene `"BLOQUEO"`, cierra las colas y el pipe y llama a `exit(0)`.

Cuando llega entrada por teclado (`POLLIN` en `fds[0]`):

- Lee el número de opción con `scanf`.

- Llama a `procesar_opcion(opcion)`.

Acceso al fichero de cuentas

Dos funciones gestionan la lectura y escritura en `cuentas.dat`:

`leer_cuenta(id, c)` — búsqueda lineal: abre en modo `"rb"` y recorre las estructuras con `fread` hasta encontrar el `numero_cuenta` buscado.

`escribir_cuenta(c)` — abre en modo `"rb+"`, recorre el fichero registrando la posición con `ftell` implícito (mediante el contador `pos`) hasta encontrar la cuenta, hace `fseek(f, pos, SEEK_SET)` y sobreescribe con `fwrite`. Esta combinación es equivalente a usar el offset calculado como `(numero_cuenta - ID_INICIAL) * sizeof(Cuenta)`, pero funciona también si el orden físico en el fichero no coincide exactamente con el rango de IDs (p. ej. tras borrados).

Threads bancarios

Cada operación se delega a un thread POSIX creado con `pthread_create()` y esperado inmediatamente con `pthread_join()` (operación síncrona desde el punto de vista del menú). La función `lanzar_operacion(fn, d)` encapsula este patrón.

Todas las funciones de thread siguen el mismo esquema:

1. Abrimos el semáforo `/sem_cuentas` con `abrir_sem_cuentas()`.
2. `sem_wait(sem)` — inicio de sección crítica.
3. `leer_cuenta()` → modificar saldos → `escribir_cuenta()`.
4. `sem_post(sem)` — fin de sección crítica.
5. `sem_close(sem)`.
6. `enviar_log()` y, cuando aplica, `enviar_monitor()`.
7. `free(d)`.

`thread_deposito` — suma `cantidad` al saldo de la divisa elegida.

`thread_retiro` — comprueba saldo suficiente y que `cantidad <= límite_retiro_divisa` (de `config.txt`). Solo resta el saldo si ambas condiciones se cumplen.

`thread_transferencia` — carga las dos cuentas (origen y destino) dentro de la misma sección crítica. Comprueba saldo y límite de transferencia. Si todo es

correcto, resta en origen, suma en destino y escribe ambas cuentas antes de liberar el semáforo.

`thread_mover_divisa` — convierte el importe de divisa origen a EUR (dividiendo por el tipo de cambio correspondiente) y de EUR a divisa destino (multiplicando). Resta de origen y suma en destino dentro de la sección crítica.

`consultar_saldos()` — no es un thread, pero también adquiere `/sem_cuentas` para leer la cuenta con consistencia. Muestra los tres saldos y el total convertido a EUR: `total = saldo_eur + saldo_usd/CAMBIO_USD + saldo_gbp/CAMBIO_GBP`.

Comunicación con padre y monitor

`enviar_log(tipo_op, cantidad, divisa, estado)` — rellena un `DatosLog` con el estado de la operación y lo manda por `MQ_LOG`. El padre lo recibirá, lo formateará y lo escribirá en `transacciones.log`.

`enviar_monitor(cuenta_origen, cuenta_destino, tipo_op, cantidad, divisa)` — rellena un `DatosMonitor` y lo manda por `MQ_MONITOR`. El proceso monitor lo procesará para detectar anomalías.

3.4 `monitor.c` — Proceso de vigilancia

Proceso independiente lanzado por `banco.c` al inicio del sistema. Termina al recibir `SIGTERM` del padre. Ignora `SIGINT` para que Ctrl+C en el terminal no lo mate accidentalmente.

Inicialización

1. Registra el manejador de `SIGTERM` que activa `g_salir`.
2. Lee `config.txt` con `leer_config_monitor()`, extrayendo `UMBRAL_RETIROS` y `UMBRAL_TRANSFERENCIAS`.
3. Abre `MQ_MONITOR` (modo `O_RDONLY`) y `MQ_ALERTA` (modo `O_WRONLY`).

Bucle de análisis

Usa `poll()` con timeout de 500 ms sobre el descriptor de `MQ_MONITOR`. Cada vez que llega un mensaje llama a `analizar()`.

`analizar(dm)` — aplica dos análisis independientes sobre cada mensaje recibido:

1. **Retiros consecutivos:** mantiene un array `g_retiros[]` de `TrackRetiros` (uno por cuenta). Cada retiro incrementa el contador `retiros_consecutivos`. Si alcanza `g_umbral_retiros`, envía una alerta `ALERTA_RETIROS` y resetea el contador. Cualquier operación que no sea un retiro resetea el contador de esa cuenta.

2. **Transferencias repetitivas:** mantiene un array `g_transf[]` de `TrackTransferencias` (uno por par origen-destino). Cada transferencia entre el mismo par incrementa su contador. Si alcanza `g_umbral_transferencias`, envía `ALERTA_TRANSFERENCIAS` y resetea.

`enviar_alerta(cuenta_id, tipo_alerta)` — rellena un `DatosAlerta` con el `cuenta_id` y el tipo de alerta formateado como `"RETIROS_EXCESIVOS en cuenta XXXX"` y lo envía por `MQ_ALERTA`.

---

4. Mecanismos de IPC y sincronización

4.1 Semáforos con nombre POSIX

Se usan dos semáforos con nombre, inicializados con valor 1 (comportamiento de mutex):

| Nombre | Recurso protegido | |--------|----------| | `/sem_cuentas` | Lectura/escritura en `cuentas.dat` | | `/sem_config` | Lectura y actualización de `PROXIMO_ID` en `config.txt` |

`banco.c` los crea con `sem_open(..., O_CREAT|O_EXCL, 0600, 1)` durante la inicialización. Los hijos y el padre los abren con `sem_open(..., 0)` (sin crear) cuando los necesitan. Al cierre del sistema, `banco.c` los destruye con `sem_unlink()`.

El uso de semáforos con nombre (en lugar de semáforos en memoria compartida o mutexes de proceso) permite que procesos independientes lanzados con `execv()` puedan adquirirlos sin necesidad de heredarlos a través de la memoria del proceso padre.

4.2 Colas de mensajes POSIX

Se crean tres colas con `mq_open()` antes de lanzar ningún proceso hijo:

| Cola | Dirección | Tipo de mensaje | Capacidad | |------|----------|----------------|---------| | `MQ_MONITOR` (`/securebank_monitor`) | hijo → monitor | `DatosMonitor` | 10 mensajes | | `MQ_LOG` (`/securebank_log`) | hijo → padre | `DatosLog` | 10 mensajes | | `MQ_ALERTA` (`/securebank_alerta`) | monitor → padre | `DatosAlerta` | 10 mensajes |

El padre las crea con `O_RDWR|O_NONBLOCK` para poder drenarlas sin bloqueo. Los hijos las abren con `O_WRONLY` solo para escritura. El monitor abre `MQ_MONITOR` con `O_RDONLY` y `MQ_ALERTA` con `O_WRONLY`.

El descriptor de una cola POSIX es un entero que puede usarse directamente como `fd` en `poll()`, lo que permite al padre y al monitor integrar la espera sobre colas en el mismo mecanismo de multiplexado que usan para el resto de eventos.

4.3 Pipes anónimos

Se crea un pipe anónimo por cada sesión de usuario activa. El extremo de escritura lo conserva el padre; el de lectura lo hereda el hijo en el `fork()` y lo pasa a `execv()` como argumento (número de descriptor en texto).

El pipe es unidireccional (padre → hijo) y solo se usa para comunicar eventos críticos: mensajes de bloqueo (`"BLOQUEO: ..."`). El hijo lo vigila en su `poll()` como segundo descriptor. Cuando el padre detecta que debe bloquear una cuenta, escribe en el pipe y cierra el extremo de escritura, lo que provoca que el hijo obtenga `POLLIN` y detecte el mensaje.

4.4 Señales

| Señal | Quién la recibe | Efecto | |-------|----------|--------| | `SIGTERM` | `banco.c` | Activa `g_salir = 1`, inicia cierre controlado | | `SIGINT` | `banco.c` | Igual que `SIGTERM` | | `SIGTERM` | `monitor.c` | Activa `g_salir = 1`, el monitor termina limpiamente | | `SIGINT` | `monitor.c` | Ignorada (`SIG_IGN`) | | `SIGCHLD` | `banco.c` | `SIG_DFL` — permite que `waitpid` recoja el estado |

4.5 Threads POSIX (pthreads)

Dentro de cada proceso hijo, cada operación bancaria se ejecuta como un thread separado creado con `pthread_create()`. La función `lanzar_operacion(fn, d)` crea el thread y lo une inmediatamente con `pthread_join()`, haciendo la ejecución efectivamente síncrona desde el punto de vista del menú. Este diseño permite escalar fácilmente a operaciones asíncronas en el futuro (pool de threads con semáforo de conteo para `NUM_HILOS`).

El acceso al fichero compartido `cuentas.dat` desde los threads está protegido por `/sem_cuentas`, garantizando la exclusión mutua incluso entre threads de distintos procesos.

---

5. Flujo de ejecución completo

```
banco arranca
│
├─ leer config.txt
├─ crear /sem_cuentas, /sem_config
├─ crear cuentas.dat (si no existe)
├─ crear MQ_MONITOR, MQ_LOG, MQ_ALERTA
├─ fork+execv → monitor
│       monitor: abre MQ_MONITOR (R), MQ_ALERTA (W)
│               bucle poll(MQ_MONITOR, 500ms)
│
└─ bucle login (poll: stdin, MQ_LOG, MQ_ALERTA)
   │
   ├── usuario introduce 0 (nueva cuenta)
   │       banco: sem_wait(/sem_config)
   │               lee PROXIMO_ID → ID=1001
   │               PROXIMO_ID=1002 → guarda en config.txt
   │               sem_post(/sem_config)
   │       banco: sem_wait(/sem_cuentas)
   │               fwrite(Cuenta) → cuentas.dat
   │               sem_post(/sem_cuentas)
   │       banco: fork+execv → usuario (cuenta 1001, pipe_rd=5)
   │               usuario: abre MQ_LOG(W), MQ_MONITOR(W)
   │                        poll(stdin, pipe_rd)
   │                        mostrar menú
   │
   ├── usuario elige opción 1 (Depósito)
   │       usuario: calloc(DatosOperacion), pedir_divisa, scanf(cantidad)
   │               pthread_create(thread_deposito)
   │                   sem_wait(/sem_cuentas)
   │                   leer_cuenta → saldo_eur += cantidad
   │                   escribir_cuenta
   │                   sem_post(/sem_cuentas)
   │                   mq_send(MQ_LOG, DatosLog)
   │                   mq_send(MQ_MONITOR, DatosMonitor)
   │               pthread_join
   │
   ├── banco recibe DatosLog en MQ_LOG
   │       procesar_log(): formatea línea → escribir_log()
   │       "[2026-04-14 10:00:00] Deposito en cuenta 1001: 500.00 EUR - OK"
   │
   ├── usuario elige opción 2 (Retiro) ×3 consecutivos
   │       monitor recibe 3 DatosMonitor con OP_RETIRO
   │               analizar(): retiros_consecutivos → 3 == UMBRAL_RETIROS
   │               mq_send(MQ_ALERTA, "RETIROS_EXCESIVOS en cuenta 1001")
   │
```

```
   ├── banco recibe DatosAlerta en MQ_ALERTA
   │       procesar_alertas(): detecta ALERTA_RETIROS → bloquear
   │       escribir_log(): "[...] ALERTA: RETIROS_EXCESIVOS ... — cuenta
BLOQUEADA"
   │       buscar_hijo(1001) → pipe_wr
   │       write(pipe_wr, "BLOQUEO: RETIROS_EXCESIVOS en cuenta 1001")
   │       close(pipe_wr)
   │
   ├── usuario recibe "BLOQUEO" en pipe_rd via poll
   │       imprime: "[ALERTA DEL BANCO]: BLOQUEO: ..."
   │       mq_close + exit(0)
   │
   └── banco detecta terminación del hijo con waitpid(WNOHANG)
           retorna al bucle de login
```

---

6. Detección de anomalías

El monitor mantiene dos tablas en memoria:

Retiros consecutivos

```
typedef struct {
    int   cuenta_id;
    int   retiros_consecutivos;
    float ultimo_retiro;
} TrackRetiros;
```

- Cada mensaje `OP_RETIRO` incrementa el contador de la cuenta.
- Cualquier operación diferente (depósito, transferencia) **resetea** el contador.
- Cuando `retiros_consecutivos >= UMBRAL_RETIROS` (por defecto 3), se envía la alerta y se resetea el contador.

Transferencias repetitivas

```
typedef struct {
    int cuenta_origen;
    int cuenta_destino;
    int contador;
} TrackTransferencias;
```

- Cada mensaje `OP_TRANSFERENCIA` incrementa el contador del par (origen, destino).
- Cuando `contador >= UMBRAL_TRANSFERENCIAS` (por defecto 2), se envía la alerta y se resetea.

Decisión de bloqueo (en `banco.c`)

Al recibir una alerta, `banco.c` hace `strstr` sobre el campo `mensaje` buscando las cadenas `"RETIROS_EXCESIVOS"` o `"TRANSFERENCIAS_REPETITIVAS"`. Cualquiera de las dos desencadena el bloqueo automático: escritura del mensaje `"BLOQUEO"` en el pipe del hijo afectado.

---

7. Fichero de configuración

El fichero `config.txt` parametriza el sistema en tiempo de ejecución sin necesidad de recompilar:

```
PROXIMO_ID=1001          # Próximo número de cuenta a asignar
LIM_RET_EUR=5000         # Límite máximo por retiro en euros
LIM_RET_USD=2000         # Límite máximo por retiro en dólares
LIM_RET_GBP=3000         # Límite máximo por retiro en libras
LIM_TRF_EUR=10000        # Límite máximo por transferencia en euros
LIM_TRF_USD=10000        # Límite máximo por transferencia en dólares
LIM_TRF_GBP=10000        # Límite máximo por transferencia en libras
# Umbrales de detección de anomalías
UMBRAL_RETIROS=3         # Nº de retiros consecutivos para alerta
UMBRAL_TRANSFERENCIAS=2  # Nº de transf. repetitivas para alerta
# Parámetros de Ejecución
NUM_HILOS=4              # Máximo de hilos simultáneos (referencia)
ARCHIVO_CUENTAS=cuentas.dat
ARCHIVO_LOG=transacciones.log
# Tipo de cambio (base EUR)
CAMBIO_USD=1.08          # 1 EUR = 1.08 USD
CAMBIO_GBP=0.86          # 1 EUR = 0.86 GBP
```

`PROXIMO_ID` es el único campo que el sistema modifica en tiempo de ejecución (función `guardar_proximo_id`), siempre dentro de la sección crítica protegida por `/sem_config`.

La constante `ID_INICIAL 1001` definida en `banco_comun.h` debe coincidir con el valor inicial de `PROXIMO_ID`. Se usa para calcular el offset de `fseek()` al posicionar el puntero de fichero en `cuentas.dat`.

---

8. Compilación y arranque

Compilación

```
make
```

El `Makefile` compila los tres binarios con `-Wall -Wextra -pthread` (cero warnings) y enlaza con `-lrt` para las colas POSIX:

```
gcc -Wall -Wextra -pthread -o banco   banco.c   -pthread -lrt
gcc -Wall -Wextra -pthread -o usuario usuario.c -pthread -lrt
gcc -Wall -Wextra -pthread -o monitor monitor.c -pthread -lrt
```

Arranque

```
./banco
```

Solo hace falta lanzar `banco`. El proceso lanza automáticamente al monitor antes del bucle de login.

Limpieza

```
make clean
```

Elimina los binarios y los ficheros generados en ejecución (`cuentas.dat`, `transacciones.log`).

Ejemplo de sesión

```
+==============================+
|    SecureBank  --  Login     |
+==============================+

Introduzca numero de cuenta (0=nueva, -1=salir): 0
Nombre del titular: Alice
Cuenta 1001 creada para 'Alice'.

+==========================+
|  Cuenta 1001             |
|==========================|
| 1. Deposito              |
| 2. Retiro                |
| 3. Transferencia         |
| 4. Consultar saldos      |
| 5. Mover divisas         |
| 6. Salir                 |
+==========================+
Opcion: 1
Divisa (0=EUR, 1=USD, 2=GBP): 0
Cantidad a depositar: 1000
Deposito OK: +1000.00 EUR

Opcion: 4

+-----------------------------+
|  Saldos cuenta 1001        |
|-----------------------------|
| EUR:                1000.00 |
| USD:                   0.00 |
| GBP:                   0.00 |
|-----------------------------|
| Total (EUR):        1000.00 |
+-----------------------------+
```

Ejemplo de log generado (`transacciones.log`)

```
[2026-04-14 10:00:00] Deposito en cuenta 1001: 1000.00 EUR - OK
[2026-04-14 10:01:30] Retiro en cuenta 1001: 200.00 EUR - OK
[2026-04-14 10:02:15] Retiro en cuenta 1001: 300.00 EUR - OK
[2026-04-14 10:02:45] Retiro en cuenta 1001: 150.00 EUR - OK
[2026-04-14 10:02:46] ALERTA: RETIROS_EXCESIVOS en cuenta 1001 - cuenta
BLOQUEADA
```

9. Capturas de funcionamiento

> **Nota:** incluir en la memoria final capturas de pantalla reales de las siguientes situaciones:

1. **Arranque del sistema** — terminal mostrando el menú de login inicial.
2. **Creación de cuenta nueva** — secuencia de entrada `0`, nombre del titular, y confirmación `"Cuenta XXXX creada para 'Nombre'."`.
3. **Login en cuenta existente** — entrada de número de cuenta y mensaje de bienvenida.
4. **Menú de usuario** — pantalla del menú interactivo de operaciones.
5. **Depósito exitoso** — operación de depósito con confirmación `"Deposito OK: +X.XX DIV"`.
6. **Retiro con saldo insuficiente** — mensaje de error `"Saldo insuficiente"`.
7. **Retiro que supera el límite** — mensaje `"Supera el limite de retiro"`.
8. **Transferencia entre cuentas** — dos terminales simultáneos; verificar que el saldo cambia en ambas.
9. **Conversión de divisas** — operación de mover divisas mostrando `"Conversion OK: X.XX EUR -> Y.YY USD"`.
10. **Consulta de saldos** — tabla de saldos con total en EUR.
11. **Detección de anomalía y bloqueo** — tres retiros consecutivos; pantalla mostrando `"[ALERTA DEL BANCO]: BLOQUEO: RETIROS_EXCESIVOS en cuenta XXXX"` y terminación de la sesión.
12. **Log generado** — contenido del fichero `transacciones.log` con entradas de operaciones y alertas.
13. **Dos usuarios simultáneos** — dos terminales activos, mostrando que el sistema gestiona concurrentemente dos sesiones.

---

10. Cuaderno de bitácora

> **Instrucción del enunciado:** *"El cuaderno de bitácora debe incluir una enumeración de los problemas que se han ido encontrando en el desarrollo. Se escribirá a mano."*

A continuación se listan los principales problemas encontrados durante el desarrollo. Esta sección debe transcribirse a mano en la entrega impresa:

Problema 1 — Tamaño de mensaje en colas POSIX
Al crear las colas con `mq_attr`, si el campo `mq_msgsize` no se ajusta exactamente al tamaño de la estructura que se va a transmitir, `mq_send` devuelve `EMSGSIZE`. Fue necesario crear tres colas separadas (una por tipo de mensaje) en lugar de una cola genérica.

Problema 2 — Límite del sistema en `mq_maxmsg`
El sistema Linux por defecto limita `mq_maxmsg` a 10 mensajes (se puede verificar en `/proc/sys/fs/mqueue/msg_max`). Si se establece un valor mayor en el código, `mq_open` falla con `EINVAL`. La solución fue usar `MQ_MAXMSG = 10`.

Problema 3 — Descriptores de colas heredados por `execv`
Tras el `fork()`, el proceso hijo hereda los descriptores de las colas abiertas por el padre. Si no se cierran antes de `execv()`, el hijo mantiene referencias a colas que no debería usar (en particular, las que son exclusivas del padre). Se añadió el cierre explícito de `g_mq_log`, `g_mq_alerta` y `g_mq_monitor` en el proceso hijo antes de la llamada a `execv`.

Problema 4 — Paso del descriptor de lectura del pipe como argumento
`execv` reemplaza la imagen del proceso, por lo que no se pueden pasar variables en memoria. El descriptor de lectura del pipe (`pfd[0]`) se convierte a cadena con `snprintf` y se pasa como argumento de línea de comandos. En `usuario.c` se recupera con `atoi(argv[2])`.

Problema 5 — Acceso concurrente a `cuentas.dat` desde múltiples procesos
Con varias sesiones abiertas simultáneamente, dos threads de distintos procesos podían leer y escribir el fichero al mismo tiempo, corrompiendo datos. La solución fue usar el semáforo con nombre `/sem_cuentas`: al ser un semáforo del sistema de

nombres POSIX, es visible por todos los procesos sin necesidad de memoria compartida.

Problema 6 — Carrera en `PROXIMO_ID`
Si dos usuarios solicitaban crear cuenta simultáneamente, ambos podían leer el mismo `PROXIMO_ID` y asignar el mismo número. El semáforo `/sem_config` envuelve la operación completa de leer-incrementar-guardar en `config.txt`, eliminando la carrera.

Problema 7 — `poll()` sobre colas POSIX
Inicialmente se intentó usar `select()` sobre los descriptores de colas, lo que funcionaba en algunos kernels pero no en todos. Se cambió a `poll()`, que ofrece soporte más consistente para descriptores de colas POSIX en Linux moderno.

Problema 8 — Sincronización del cierre padre-hijo
Al usar `waitpid(..., WNOHANG)` en el bucle de sesión, el padre no bloqueaba pero tampoco podía distinguir bien cuándo el hijo terminaba por bloqueo versus por elección del menú (opción 6). Se unificó el flujo: el hijo siempre llama a `exit(0)` en ambos casos, y el padre lo detecta con `waitpid` devolviendo el PID del hijo.

Problema 9 — Monitor sin pipe, solo señal de cierre
El monitor no tiene pipe de entrada del padre. La única forma de detenerlo es mediante `SIGTERM`. Se comprobó que si el padre moría sin enviar `SIGTERM`, el monitor quedaba huérfano. La solución es el `waitpid` del monitor en el bloque de cierre de `banco.c`, precedido siempre por `kill(g_monitor_pid, SIGTERM)`.

Problema 10 — Warnings de compilación con `-Wextra`
El flag `-Wextra` detectó parámetros de señal sin usar (el entero `s` en `manejador_sigterm`). Se resolvió con la anotación `(void)s;` al inicio de los manejadores.

Fin de la memoria técnica — SecureBank

Mario Gayarre Tolivar · Rodrigo Torre Santamaría · Marco Aurelio Cabral de Pablo
Sistemas Operativos — UFV Madrid — Abril 2026